

Lightweight RPC for Kotlin Multiplatform

by

Aleksandr Stupnikov

Master Thesis in Computer Science and Software Engineering

Submission: May 18, 2026

Supervisor: Dr. A. Podkopaev

Statutory Declaration

Family Name, Given/First Name	Stupnikov, Aleksandr
Matriculation number	30008334
Kind of thesis submitted	Master Thesis

English: Declaration of Authorship

I hereby declare that the thesis submitted was created and written solely by myself without any external support. Any sources, direct or indirect, are marked as such. I am aware of the fact that the contents of the thesis in digital form may be revised with regard to usage of unauthorized aid as well as whether the whole or parts of it may be identified as plagiarism. I do agree my work to be entered into a database for it to be compared with existing sources, where it will remain in order to enable further comparisons with future theses. This does not grant any rights of reproduction and usage, however.

This document was neither presented to any other examination board nor has it been published.

German: Erklärung der Autorenschaft (Urheberschaft)

Ich erkläre hiermit, dass die vorliegende Arbeit ohne fremde Hilfe ausschließlich von mir erstellt und geschrieben worden ist. Jedwede verwendeten Quellen, direkter oder indirekter Art, sind als solche kenntlich gemacht worden. Mir ist die Tatsache bewusst, dass der Inhalt der Thesis in digitaler Form geprüft werden kann im Hinblick darauf, ob es sich ganz oder in Teilen um ein Plagiat handelt. Ich bin damit einverstanden, dass meine Arbeit in einer Datenbank eingegeben werden kann, um mit bereits bestehenden Quellen verglichen zu werden und dort auch verbleibt, um mit zukünftigen Arbeiten verglichen werden zu können. Dies berechtigt jedoch nicht zur Verwendung oder Vervielfältigung.

Diese Arbeit wurde noch keiner anderen Prüfungsbehörde vorgelegt noch wurde sie bisher veröffentlicht.

.....
Date, Signature

Abstract

A large subset of modern software for Kotlin and Kotlin Multiplatform (KMP) uses network interactions extensively. Backend microservices, client-server applications and distributed systems are typical examples. Many such projects follow *shared-codebase* (also called *monolithic*) model in which client and server live in a single Kotlin project and share data types and parts of business logic. Existing RPC frameworks for Kotlin like gRPC and Kotlin RPC are not boilerplate-heavy in general. However, they all require developer to describe remote functions in a separate service interface. This interface plays the role of a contract between client and server, which is necessary when client and server are developed independently. In shared-codebase setting this contract becomes redundant, because client and server already share the codebase and signatures of remote functions can be the contract on their own. The interface declaration then turns into *boilerplate code* that exists only because the chosen framework requires it.

In this thesis we developed a new RPC framework for shared-codebase Kotlin projects. We called this new framework *Kotlin Remote* [25]. Kotlin Remote uses Kotlin's experimental *context parameters* feature to express remote calls without service interface declaration. Context parameter of remote function carries information about target machine, which makes function callable on this machine, and at the same time appears in function signature, which makes remote calls clearly visible at the type level.

Kotlin Remote is intentionally scoped to the shared-codebase case. It does not try to compete with general purpose RPC frameworks like gRPC or Kotlin RPC outside this scope. In particular, it does not support cross-language communication and is not meant for projects in which client and server are developed independently in separate codebases. In those scenarios the service interface required by gRPC or Kotlin RPC is not redundant, so the trade made by Kotlin Remote does not pay off. Inside its target scope, however, Kotlin Remote removes the redundant interface declaration and leads to substantial reduction of boilerplate compared to existing alternatives, while keeping remote calls explicit for the developer. Shared-codebase model is very common in Kotlin and KMP environment, so we believe Kotlin Remote will be useful despite its narrow scope.

Contents

1	Introduction	1
1.1	Goal and objectives	2
2	Background and Related work	3
2.1	RPC	4
2.2	Distributed Objects	4
2.3	REST	4
2.4	gRPC	5
2.5	Kotlin RPC	6
2.6	Coeffects	6
2.7	Multi-tier programming languages	7
2.8	Kotlin Ecosystem	8
2.8.1	Kotlin Coroutines	8
2.8.2	Kotlin Multiplatform	8
2.8.3	kotlinx.serialization	9
2.8.4	Ktor	9
2.8.5	Kotlin context parameters	9
3	Kotlin Remote Implementation	10
3.1	Remote functions	10
3.1.1	Client	10
3.1.2	Server	13
3.1.3	Single codebase	14
3.1.4	Remote context relations	15
3.2	Remote functions code generation	16
3.2.1	Remote function body generation	17
3.2.2	Callable map generation	17
3.3	Exception handling	18
3.4	Architecture	20
3.5	Ktor integration	20
3.6	Remote classes	20
3.6.1	Remote classes code generation	22
3.7	Garbage collection	23
4	Evaluation of the Kotlin Remote	23
4.1	Example applications	25
4.1.1	Todo application	25
4.1.2	Rooms chat	25
4.1.3	Social platform	26
4.1.4	CMS	27
4.2	Comparison to Kotlin RPC	29
4.2.1	Per-function overhead	30
4.3	Limitations	31
4.3.1	Narrow scope of application	31
4.3.2	Dependency on experimental Kotlin features	32
4.4	Future work	32
4.4.1	Code slicing	32
4.4.2	Remote lambdas	32

4.4.3 Bidirectional streaming	32
5 Conclusions	32
A Note on Code coloring	37

1 Introduction

Nowadays, Kotlin programming language is used frequently for developing software that relies heavily on network interactions. Such software includes backend microservices, client-server applications and distributed systems. Many of these projects follow *shared-codebase* (also called *monolithic*) model in which client and server live in a single Kotlin project and share data types and parts of business logic. By shared-codebase project we mean a project where all the parts that communicate over the network are developed together, compiled together and released together. This is the setting this thesis is concerned with. The opposite setting, in which client and server are developed independently, often in different languages, by different teams, is out of scope.

Usually code base of shared-codebase applications can be roughly separated into two parts: part responsible for actual business logic and part responsible for network interactions. While the former part differs significantly from program to program, network interactions part is usually very similar for a wide variety of applications. This leads to the abundance of boilerplate code in the said part. For convenience from now on we will call part of the program source code responsible for network interactions *network part* of the program. It is desired to make network part of the program as small and simple as possible, because this would allow developers to think less about network and concentrate on the actual business logic. In shared-codebase setting this is especially important, because lots of information is essentially duplicated between client and server parts of the codebase, and a lot of this duplication can be avoided if the network part is designed with shared codebase in mind.

However, standard ways to implement network part in Kotlin are not well suited for making this part concise in shared-codebase setting. There are two main families of network technologies used in Kotlin today. The first one is general purpose HTTP frameworks, the most popular of which is Ktor. The second one is RPC frameworks, of which gRPC and Kotlin RPC are notable representatives.

Ktor forces developers to manually write routing logic, serialization and deserialization of request and response bodies, HTTP client calls and error handling code. This gives fine-grained control over the network part logic, which is beneficial when complex public APIs are exposed to the outside world. However, for internal service-to-service communication in shared-codebase projects, this level of control is unnecessary and the manual work is pure overhead.

gRPC and Kotlin RPC reduce this overhead by following *Remote Method Invocation* (RMI) pattern. Developer declares a service as an interface, implements it in a class and then receives a client proxy for that interface. These frameworks are not boilerplate-heavy in general. In fact, their per function overhead is already small: developer adds a method signature to the service interface and writes the implementation. The IDL or `@Rpc` interface plays an important role in those frameworks. It is the explicit contract between client and server, which is exactly what is needed when client and server are developed independently, in different languages or with different release schedules. In that setting the interface is not boilerplate at all, it is a necessary part of the design.

The picture changes in shared-codebase setting. Here client and server already share the codebase, data types are shared, and a single change to a signature is a compile-time change for both sides at once. The contract between client and server is implicit in the shared code. Declaring it explicitly in a service interface duplicates information that is

already there. The interface declaration, the implementation class that forwards calls to the actual logic, the IDL with its proto-to-domain mappings in case of gRPC, all become redundant. They are boilerplate not because the framework is verbose, but because they exist only to fulfill the framework's structural requirement of having a service interface, while the contract they encode is already encoded in the shared codebase.

Existing RPC frameworks have a few other drawbacks in shared-codebase setting that follow from the same root cause. Top level functions cannot be remote at all, every remote function has to be a method of some service. At the call site remote call looks identical to a local one, there is nothing in the syntax that tells developer that this is a network operation that might be slow or fail. The approach follows strict OOP style which is not the only viable option for Kotlin. These design choices make sense for gRPC and Kotlin RPC because they are general purpose frameworks aiming at much wider scope than shared-codebase projects. The same design choices are precisely what makes them carry redundant declarations in shared-codebase setting.

The goal of the thesis is to develop alternative approach to organizing network interactions in shared-codebase Kotlin projects that will make network part smaller and easier by reducing amount of boilerplate code. We deliberately scope the work to shared-codebase setting and do not try to compete with gRPC, Kotlin RPC or Ktor outside this scope. The hypothesis is that by sacrificing generality (cross-language support, public-facing API ergonomics, separate evolution of client and server) the framework can give substantial reduction of boilerplate inside its target scope.

To achieve this goal we took a look at RPC in Kotlin from another perspective not common for object-oriented languages. In this paper we explore using *coeffects*, a concept from the world of functional programming, to implement RPC framework that requires to write much less boilerplate code than more traditional RPC frameworks in shared-codebase setting. More than that, this approach makes network code easily distinguishable from the rest of the program. In Kotlin programming with coeffects can be expressed by using experimental language feature called *context parameters*, which allows function to have several receiver parameters that are implicitly passed to the function from the invocation context. We use context parameters to carry information about target machine. Such context parameter appears in remote function signature, which makes remote calls clearly visible at the type level, and at the same time is resolved automatically at the call site, so caller does not have to pass it manually.

1.1 Goal and objectives

The goal of this thesis is to develop an RPC framework specifically for Kotlin Multiplatform shared-codebase projects that reduces boilerplate code compared to existing alternatives in this setting. The framework is not meant to replace general purpose RPC frameworks outside of shared-codebase scope.

To reach this goal we set the following objectives:

1. Prototype RPC framework that uses context parameters as the mechanism for expressing remote calls.
2. Prototype support for distributed objects, that is classes with state that lives on remote machine.
3. Implement the framework as a Kotlin compiler plugin together with a runtime library.

4. Evaluate the result by comparing example projects written using developed framework with alternatives.

This work has the following structure:

1. Section 2 reviews background and related work on organizing network interactions both in Kotlin ecosystem and outside that.
2. Section 3 describes architecture of the developed framework.
3. Section 4 evaluates the framework by using small applications and comparing them with standard approaches; discusses limitations and future work.
4. Section 5 concludes.

2 Background and Related work

This section gives an overview of RPC as whole, as well as specific frameworks used in Kotlin ecosystem to organize network interactions. Concepts used in this thesis, like *coeffects* and Kotlin context parameters, are also explained here.

Before going further it is worth restating the scope. Existing frameworks discussed below (Ktor, gRPC, Kotlin RPC) are general purpose tools that target a much wider scope than this work. They are used in cross-language projects, in projects where client and server live in separate codebases and have separate release cycles, and in projects where client and server are developed by different teams. We do not claim that Kotlin Remote competes with them in those scenarios. On the contrary, in those scenarios gRPC and Ktor are very likely the right choice. The discussion below is about how these frameworks behave in shared-codebase Kotlin projects, which is the only setting Kotlin Remote targets.

Proposed approach is a specific version of RPC. Because of that it makes sense to compare it mostly to other RPC approaches used in Kotlin. The most popular ones are gRPC and new Kotlin RPC. These frameworks are not boilerplate-heavy in general, their per function overhead is small. They do however require developer to describe remote functions in a separate service interface and to implement this interface in a service class. In shared-codebase setting both of these become redundant, because client and server already share the codebase and the contract that the service interface is supposed to encode is implicit in the shared code. Outside of shared-codebase setting the service interface is not redundant, it is the explicit contract between independently developed parts and existing frameworks are well-designed around it. Additionally, basically all the existing RPC frameworks for Kotlin use Remote Method Invocation (RMI), the object-oriented programming analog of RPC, where only methods of services can be executed remotely. Top level functions cannot. This fits Kotlin only partially, because Kotlin is not entirely OOP language. The main aim of developed framework is removal of the service interface redundancy in shared-codebase Kotlin projects.

Apart from RPC frameworks, REST architectural style is also widely used in Kotlin to organize network interactions. The most popular library for that is Ktor. Therefore, it is useful to take a brief look at REST as well. Apart from looking at existing frameworks, this section also takes a look on related ideas from functional programming. Namely on *coeffects*, that are foundational concept on which Kotlin Remote is built. *Coeffects* are tightly related to *implicit parameters*, which Kotlin recently got an implementation of in

form of experimental *context parameters* feature. Coeffects also have close ties to *multi-tier programming languages*, in which separation between different machines is encoded in the type system. Multi-tier languages are particularly relevant because they target precisely the shared-codebase setting and inspired some of the design decisions made in Kotlin Remote. All of these ideas will be discussed below.

2.1 RPC

Even though the concept of RPC is well known in the community, it is still better to provide some understanding of RPC that is used in this work.

Remote procedure calls (RPC) — a communication protocol which allows an application executing functions on a remote machine (commonly using a shared computer network) as if these functions were executed on the machine where the application is running.

Remote function — function that can be executed on a remote machine.

The goal of RPC is to hide details of network communication from an application and make remote functions calls look like ordinary function calls. This paradigm allows programmer to think less about network details and more about core logic of the application.

The idea of RPC is not new and dates back 1970s [30] with enterprise implementations appearing roughly in the 1980s [1].

It is worth noticing that RPC paradigm has been criticized for trying to hide the complexities of network communication from the developer, which can lead to wrong assumptions about latency and failure modes of remote calls [26]. We are aware of this criticism and as it will be shown later, our framework allows to make remote calls visible at the call site on a type level, while still keeping the boilerplate small.

2.2 Distributed Objects

RPC does not translate well in object-oriented world, where interactions between objects residing on different machines are required. That is why when object-oriented languages became popular specific form of RPC was developed. This form is called *RMI*, and it is a way to build communication between *distributed objects* [31] [27].

Distributed Objects — objects that can be accessed from remote machines.

RMI (remote method invocation) — communication protocol which allows an application executing methods of distributed objects on a remote machine.

Most classic implementations of RMI are CORBA, Java RMI [9] and COM/DCOM. They share lots of architectural similarities and follow the same general idea of providing distributed objects on a type level [20]. However, classic forms of RMI in their fullness are not popular nowadays. These implementations are not used in a modern production code. Modern approaches try to be more lightweight and explicit, they do not try to hide network interactions completely from the developer [23]. Developed framework follows this trend.

2.3 REST

REST (Representational State Transfer) is an architectural style for designing networked applications introduced by Roy Fielding in his PhD thesis [6]. It is built on top of HTTP

and uses HTTP methods (GET, POST, PUT, DELETE, etc.) to operate on resources identified by URLs. REST does not specify any particular serialization format, but JSON is the most common one in practice. REST is the most popular way of organizing network interactions in Kotlin today, and is usually implemented using Ktor framework [16] which will be discussed later in section 2.8.4.

In contrast with RPC, REST is centered around resources and not around procedures. From developer's point of view this means that for each new endpoint user has to design URL of the resource, choose HTTP method that should be used to access it, write routing code on the server, write serialization and deserialization logic for request and response bodies and make HTTP request from the client. All of this can be quite tedious especially when client and server share the same codebase, because in this case lots of information that is encoded in URL and HTTP method is essentially duplicated.

REST gives developer a lot of fine-grained control over network interactions. This is beneficial when network communication is exposed to the outside world (for example, public API of a service). However, when network is used for internal communication between microservices that share the same codebase, this fine-grained control is rarely needed and boilerplate cost dominates.

2.4 gRPC

gRPC [8] is one of the most well-known modern RPC frameworks. It was developed at Google as an open-source successor to internal RPC system called Stubby [23], which was used internally for more than a decade to connect microservices in Google data centers. Today gRPC supports a wide variety of programming languages, including Kotlin.

Communication in gRPC happens over HTTP/2 and uses Protocol Buffers [22] as a default serialization format. Programmer describes services in a special interface definition language (IDL). For each described service gRPC tooling generates client and server stubs in target programming language. Then user implements server stub on the server side and uses client stub on the client side to make remote calls. Stubs here play the same role as stubs in Kotlin RPC, which were briefly mentioned in the section 3.1.1.

gRPC is a mature framework with little boilerplate per remote function in its target setting. For each remote function developer writes its description in IDL once and provides implementation on the server side. The IDL has a clear purpose: it is the cross-language contract between client and server and it is precisely what allows gRPC to support clients and servers written in different languages. In setting gRPC is designed for, the IDL is not boilerplate, it is the framework's most useful feature.

In shared-codebase Kotlin setting, however, several gRPC features become redundant. Separate IDL duplicates type information that is already shared between client and server through the codebase. For each remote function developer has to write its description in IDL, implement service method that calls actual function and provide mappings between IDL types and domain types of the application. The latter is the most visible cost, because in shared-codebase project domain types already exist in the shared code and have to be mapped to and from generated IDL types just for the sake of going over the wire. Cross-language support, the feature that justifies all of this in the general case, is not used at all when both sides are Kotlin.

Two more drawbacks are not specific to shared-codebase setting but matter for our dis-

cussion. First, every remote function should be a method of some service, top level functions cannot be remote. Second, remote call on the client side looks indistinguishable from local call. This is not entirely good, because remote calls are inherently slower and more error-prone than local ones, so developer should be aware of when remote call is happening.

It is worth noticing that gRPC follows RMI approach: every remote function is a method of some service, services are first-class entities. This fits naturally with strict OOP languages like Java, but for Kotlin it is not the only option, as we will show later in this thesis.

2.5 Kotlin RPC

Kotlin RPC (also known as `kotlinx.rpc`) [14] is a relatively new RPC framework from JetBrains targeted specifically at Kotlin and Kotlin Multiplatform. The general idea of Kotlin RPC is similar to gRPC, but it is more idiomatic for Kotlin. Instead of separate IDL, user declares service as a Kotlin interface annotated with `@Rpc`. Compiler plugin then generates client stub for this interface and provides means to expose its implementation on the server side. The exact example of Kotlin RPC usage is shown later in section 3.1.1 on listing 2.

Comparing to gRPC, Kotlin RPC is less verbose for shared-codebase scenarios, because there is no need to write IDL and mapping between IDL types and domain types of the application. Kotlin RPC is in fact quite a thin framework: for each remote function developer writes a method signature in `@Rpc` interface and an override in the implementation class. There is almost no other per function overhead. The framework is well-designed and not boilerplate-heavy in any general sense.

What remains, however, is the `@Rpc` interface itself together with its implementation class. As in gRPC, this interface is the contract between client and server. In separate-codebase project it is a useful thing to have: it is the type that survives both sides being changed independently. In shared-codebase Kotlin project the contract is implicit in the shared code and the `@Rpc` interface duplicates it. Implementation class on the server only forwards calls to the actual logic, which in shared-codebase project lives in the same module anyway, so this class is essentially a layer of indirection that exists only because Kotlin RPC requires it. The same fundamental drawbacks as in gRPC also remain: top level functions cannot be remote, every remote function has to be a method of some service and remote calls are not visible at the call site. Just like gRPC, Kotlin RPC is essentially an RMI framework which is not the only viable approach for Kotlin.

2.6 Coeffects

Approach used in Kotlin Remote framework is heavily inspired by ideas from the world of functional programming. Specifically by the concept of *coeffects* [19][3].

Coeffect — property of a computation that describes what this computation requires from its environment in order to be executed.

Coeffects can be thought of as a dual to *effects*. While effects describe what computation does to its environment (for example, mutating a global variable, throwing an exception or performing IO), coeffects describe what computation requires from its environment to be executed (for example, access to specific resource, presence of certain configuration value or, in our case, way to reach remote machine). Petricek et al. [19] introduced

unified static analysis for context-dependence that captures several seemingly different examples like liveness analysis, implicit parameters and dataflow caching under one umbrella. Theoretical foundation of coeffects relies on indexed comonads, but in practice coeffects can be expressed using more familiar language constructs.

It is useful to look closer at duality between effects and coeffects. Effects in their modern form are usually expressed via *algebraic effects* [21] and *effect handlers*. Algebraic effects describe operations that computation can perform on its environment (like throwing an exception, reading from a file or sending a network request) and effect handlers describe how exactly these operations should be interpreted. Languages like Koka [17] put algebraic effects in the heart of their type system. The duality is the following: effects describe what computation does to the environment and how this is interpreted, while coeffects describe what computation requires from the environment and how this requirement is satisfied. In the context of this thesis remote call can be seen both as an effect (it performs network IO) and as a coeffect (it requires a way to reach remote machine). In Kotlin Remote we focus on the coeffect side of remote calls, because it allows us to express remote calls as a normal Kotlin functions with implicit context parameter, without introducing any special effect system.

The most well-known practical encodings of coeffects in functional languages are *implicit parameters* [18] and *Reader monad*. Implicit parameters are parameters of the function that are not passed explicitly at the call site, but are resolved automatically from the surrounding context. Reader monad threads read-only environment through computation in a way that does not require explicitly passing it to every function. From the user point of view both mechanisms allow expressing context-dependence of a computation in a type-checked way.

Reader monad is used by RPC framework called Servant [24] for Haskell programming language to implement remote calls. In Servant remote functions return value wrapped into `ClientM` which is basically Reader monad providing remote configuration. Concrete server is provided to remote functions by `runClientM` function. This makes remote calls in Servant clearly remote on a type level, allows top level functions to be remote and there is no need to declare interface for remote functions. Detailed example of Servant usage is presented later in section 3.1.1.

The same principle, when applied to Kotlin, gives us framework that is presented in this thesis. The role of Reader monad or Haskell-style implicit parameters in Kotlin Remote is played by Kotlin *context parameters*, which are described later in section 2.8.5. Context parameter of remote function expresses a coeffect: it states that to execute this function caller has to provide a way to reach remote machine. Such coeffect is visible in the function signature and, just like in Servant, can be satisfied either by configured remote context (and then remote call is performed) or by local context (and then function body is executed locally on the server).

2.7 Multi-tier programming languages

Idea of distinguishing on a type level where particular piece of code is executed (client or server) is not new and was explored by several programming languages designed specifically for web applications. Such languages are often called *multi-tier* or *tierless*, because they allow writing client and server code in the same language and same codebase, while still keeping clear separation between the two on a type level. Multi-tier languages are

tightly related to coeffects discussed in the previous section: location of code execution is essentially a coeffect, because computation requires from environment a particular machine to be executed on.

Examples of such languages include Links [4], Ur/Web [2] and Opa [28]. In Links, for example, function annotation can specify whether function should be executed on a client or on a server. Compiler then checks that all the calls between client and server functions are done correctly and generates code that handles network communication automatically. Ur/Web takes similar approach but additionally provides type-safe SQL queries and HTML templates. Opa goes even further by combining client, server and database code in a single language with unified type system.

Approach taken in Kotlin Remote can be seen as a library-level approximation of what multi-tier languages do at the language level. Instead of introducing new language with dedicated type system for tiering, we use existing language feature (context parameters) to encode location of code execution on a type level. Of course, this approach is less powerful than full multi-tier language, because it is limited by what can be expressed using context parameters. But it has its own advantages: it can be used in existing Kotlin projects without rewriting them in a new language and it integrates naturally with the rest of Kotlin ecosystem.

2.8 Kotlin Ecosystem

In the framework developed as part of this thesis many technologies from Kotlin Ecosystem are used. Therefore, it makes sense to quickly introduce this technologies.

2.8.1 Kotlin Coroutines

Coroutines [5] is a Kotlin feature for asynchronous programming. Functions that can be paused and resumed without blocking the executing thread are called *suspend functions* in Kotlin and have a special `suspend` modifier. Suspend functions can only be called from other suspend functions or from special coroutine builders like `runBlocking` or `launch`. This restriction is sometimes called *function coloring* problem [29][10], because suspend functions and ordinary functions are basically two distinct kinds (or *colors*) of functions in Kotlin. For our purposes function coloring is actually beneficial, because remote calls are inherently asynchronous, so it is natural for remote functions in Kotlin Remote to be suspend. Internally coroutines are implemented using continuations. Suspend functions are transformed by the Kotlin compiler into state machines that take continuation as an additional parameter.

2.8.2 Kotlin Multiplatform

Kotlin Multiplatform (KMP) [13] is a technology that allows writing code that can be compiled to several platforms, including JVM, JavaScript, Native (iOS, Linux, macOS, Windows) and WebAssembly. Kotlin code that targets KMP is split into several so-called *source sets*. There is a common source set with code shared between all the platforms and several platform-specific source sets with code that is specific to particular platform.

KMP introduces `expect/actual` mechanism for declaring code that has different implementation for different platforms. In common source set developer declares function or class with `expect` keyword, this declaration has no body. In each platform-specific source

set developer provides actual implementation of this declaration using `actual` keyword. When common source set is compiled for particular platform, `expect` declaration is replaced with corresponding `actual` one.

One important limitation of KMP is that it does not support fully fledged reflection in common source set. JVM target supports rich reflection API, but other targets either do not support reflection at all or support only very limited subset of it. This means that libraries that target KMP can not rely on reflection to gather information about types and functions at runtime. Instead such information has to be gathered at compile time, usually using compiler plugins. Kotlin Remote targets KMP and faces this limitation, which shaped a lot of architectural decisions described later in section 3.2.

2.8.3 `kotlinx.serialization`

`kotlinx.serialization` [15] is the standard library for serialization in Kotlin. It supports several formats including JSON, CBOR and Protocol Buffers. Serialization logic for user defined classes is generated by Kotlin compiler plugin from class declaration marked with `@Serializable` annotation. This means that serialization works without reflection and as a consequence works in all Kotlin Multiplatform targets. Kotlin Remote uses `kotlinx.serialization` to serialize and deserialize remote function arguments and return values.

2.8.4 Ktor

Ktor [16] is a framework for building asynchronous servers and clients in Kotlin. It is built on top of Kotlin Coroutines and is a part of Kotlin ecosystem maintained by JetBrains. Ktor follows plugin-based architecture: most of the functionality (authentication, content negotiation, logging, etc.) is provided by plugins that user installs explicitly. Kotlin Remote uses Ktor as a default transport: `core-ktor` module provides Ktor plugin called `KRemote` that handles remote calls on the server side and integration of `HttpClient` for the client side. Although Ktor is used as a default transport, the core of Kotlin Remote is transport-agnostic, alternative transports can be plugged in by providing custom implementation of `RemoteClient` interface.

2.8.5 Kotlin context parameters

Context parameters [11] is an experimental Kotlin language feature that introduces implicit parameters to Kotlin. Function with context parameters declares its requirements from the surrounding context using special `context(...)` block before function declaration. Listing 1 shows simple example of function with context parameter.

In this example function `greet` has context parameter `logger` of type `Logger`. When `greet` is called, `logger` is not passed explicitly, instead it is resolved automatically from the enclosing `context(...)` block. If there is no such block in the lexical scope of the call, compilation error is raised. Context parameters can be of any type and there can be several of them. They are similar to implicit parameters from Scala or Haskell [18], and just like them allow to express coeffects in Kotlin in a typed way.

In Kotlin Remote context parameters are used to provide remote functions with information about target machine. This way information about remote configuration is propagated through the call chain implicitly, while at the same time being visible in the type signature


```

context(logger: Logger)
fun greet(name: String) {
    logger.info("Hello, $name")
}

fun main() {
    val logger = Logger()
    context(logger) {
        greet("World")
    }
}

```

Figure 1: Function with context parameter

of remote function. Detailed description of how exactly context parameters are used in Kotlin Remote is provided later in section 3.1.1.

3 Kotlin Remote Implementation

Framework that was developed in thesis is called *Kotlin Remote*. And from now on we will refer to that this way.

3.1 Remote functions

Function that can be executed on remote machine (aka remote functions) are the integral part of any RPC framework.

In order to reason about remote functions two terms should be introduced: client and server. In RPC world *client* is machine from which remote call is made and *server* is target machine of this remote call, machine where call is actually executed.

3.1.1 Client

Let us take a look at how remote call is done on the client in Kotlin Remote. In order to call a function on a remote machine it is necessary to pass to this function some information specific to remote call, including address of the target machine and a way to serialize function arguments and deserialize its result. In existing RPC frameworks this information is provided to the function using stub. To better understand what stub is and how it provides necessary information for remote call, let us take a look at the example for Kotlin RPC in the figure 2. User have to create instance of `KtorRpcClient`, which is configured with remote host URL and serialization schema. Then using this client user can create service stub `client.withService<PizzaShop>()`, where `PizzaShop` is interface which implementation is generated automatically by Kotlin RPC. Such an interface is usually called *service* which remote machine provides.

Stub `pizzaShop` use client with which stub was created to make remote calls. It has some drawbacks:

1. Top level functions can not be called using this approach
2. User have to declare interface just for remote functions

```

@Rpc
interface PizzaShop { suspend fun orderPizza(pizza: Pizza): Receipt }
fun main() = runBlocking {
    val client: KtorRpcClient = HttpClient { ... }.rpc {
        url { host = "localhost" }
        serialization { json() }
    }

    val pizzaShop: PizzaShop = client.withService<PizzaShop>()

    pizzaShop.orderPizza(Pizza("Pepperoni"))
}

```

Figure 2: Remote function call in Kotlin RPC

3. When calling methods of the stub user does not know that calls results in network interaction, it is completely implicit
4. When user wants to call remote functions of the same service on several target machines, several stubs have to be created

gRPC uses virtually the same approach as Kotlin RPC but a bit more cumbersome.

But stubs is not the only way to provide remote function with information it needs. Alternative approach is used in functional programming. Let us take a look at remote calls in RPC framework for Haskell programming language named Servant [24].

```

position :: Int -> Int -> ClientM Position
hello    :: Maybe String -> ClientM HelloMessage

queries :: ClientM (Position, HelloMessage)
queries = do
    pos <- position 10 10
    message <- hello (Just "servant")
    return (pos, message)

run :: IO ()
run = do
    Right (pos, message) <- runClientM queries ... "localhost"
    print pos
    print message

```

Figure 3: Remote function call in Servant

Here `position` and `hello` are remote functions. We can see their return type is `ClientM x`, where `x` is the actual value function compute and `ClientM` is a monad. Inside `queries` function several calls to remote functions are made. The place where information about target machine is provided is actually `runClientM queries ... "localhost"`. `ClientM` is basically a Reader monad, because it provides configuration for remote functions to run.

This functional RPC approach solves some of the problems of OOP approach that uses stubs. Top level functions now can be called, there is no need to declare interface, all

the remote functions are clearly remote on a type level and there is no need to create several stubs to call the same functions on several target machines. Resolution of these problems are important for Kotlin because Kotlin is not solely OOP language, it has a lot of attributes of functional and procedure languages. So we believe RPC framework for Kotlin will benefit greatly by solving stated problems.

There is an alternative to Reader monad applicable to Kotlin that is called *implicit parameters*. These are parameters of the function that are resolved from the function invocation context and should not be passed explicitly when function is called in contrast to normal parameters. New language feature for Kotlin called *context parameters* [11] is basically implementation of implicit parameters for Kotlin. Using this feature it is possible to develop RPC framework for Kotlin that uses functional approach to make remote calls. That is exactly what we did in Kotlin Remote.

Let us imagine we want to execute some function `multiply` remotely. Here is client implementation for it in the figure 4.

```
fun pass(): Nothing = error("pass")
fun multiply(lhs: Long, rhs: Long): Long { pass() }
```

Figure 4: Client part of multiply function

Body of the function is basically undefined, because it is not needed on client. To pass to `multiply` function a way to communicate with remote machine, we can add context parameter to `multiply`, like on figure 5. Reader may notice that it is inconvenient to write `c.client.call(...)` for every remote function. This problem will be addressed later in section 3.2 when code generation done by the framework will be discussed. On listing 5 remote client contains all the information needed for remote call, like target address and serialization schema. Default implementation for `RemoteClient` uses *Ktor* [16] `HttpClient`, which is setup by the user. In order to call `multiply` user can write code from figure 6.

```
interface RemoteConfig {
    val client: RemoteClient
}

context(c: RemoteConfig)
suspend fun multiply(lhs: Long, rhs: Long): Long {
    return c.client.call<Long>(
        RemoteCall("multiply", arrayOf(lhs, rhs))
    )
}
```

Figure 5: Multiply function with context

The setup of `HttpClient` is quite standard, the interesting part is `remoteSerializersModule` function. This function declares serialization schema for remote function parameters and result. It returns instance of `SerializationModule` from `kotlinx.serialization` [15]. This `SerializationModule` has serializers to serialize and deserialize remote function parameters and return values. `RemoteClient.call` accepts `RemoteCall` from figure 7. Field `callableName` is the name of the function which is called and `arguments` is arguments with which this function is called. This class has a custom serializer that

```

data object CalculatorRemoteConfig : RemoteConfig {
    override val client: RemoteClient = HttpClient {
        defaultRequest { url("http://localhost:8080") ... }
        install(ContentNegotiation) {
            json(Json {
                serializersModule = remoteSerializersModule {
                    callableMap = CallableMap(...)
                }
            })
        }
    }.remoteClient("/call")
}

fun main() = runBlocking {
    context(CalculatorRemoteConfig) {
        println(multiply(6, 1))
    }
}

```

Figure 6: Remote function call

serializes arguments: `Array<Any?>` using `callableName`. This serializer is created by `remoteSerializersModule`. To implement this custom serializer information about remote function with name `callableName` is needed. The only way to get this information in Kotlin Multiplatform (*KMP*) is by asking user to provide it or gathering it automatically somehow. The problem is this information cannot be gathered at runtime because KMP (unlike Java or Kotlin JVM) does not support fully fledged reflection. This means that we cannot get function signature at runtime or type by its name, etc. But this is exactly information needed for serialization. For now, we will think that required information is gathered somehow and stored inside structure called `CallableMap`. This is basically a map from fully qualified name of remote function to information about this function which is represented by `RemoteCallable` from figure 7.

The pipeline is the following: `remoteSerializersModule` creates serializer for `RemoteCall` using `CallableMap`. This serializer uses `CallableMap` and `RemoteCall.callableName` to determine types of parameters in `RemoteCall.arguments` and serialize them. On the server `RemoteCall` is deserialized in a similar manner. Server implementation will be discussed in details in the next section.

The simplest way to get `CallableMap` it is to ask user to write it manually. Let us think for now that user has to provide information about all the remote functions by describing `CallableMap`. This problem will be addressed later in section 3.2.2.

3.1.2 Server

It was described how to use remote functions from the client side. Now it is time to understand how to handle remote calls on the server side. When remote function is called from the client HTTP request is sent to the server. For example from figure 6 HTTP will be sent to address with URL `"http://localhost:8080/call"` there will be no query parameters and payload will be serialized version of `RemoteCall`. For this specific example it will be in JSON format. Here it is worth noticing that even though the framework was

```

class CallableMap(val callableMap: Map<String, RemoteCallable> = mapOf())

class RemoteCall(
    val callableName: String,
    val arguments: Array<Any?>,
)

data class RemoteCallable(
    val returnType: KType,
    val parameterTypes: Array<KType>,
)

```

Figure 7: RemoteCall and RemoteCallable class

written with HTTP in mind, any transport protocol could be used. To do that alternative implementation for RemoteClient interface should be provided.

User could write handler for such a request himself. Either using Ktor or anything else. But to make this process easier, framework provides default handlers for remote calls. To use it Ktor server with installed KRemote plugin should be created. This can be done by writing code from figure 8. This will run HTTP server on URL "<http://localhost:8080/call>". Just like in client CallableMap is used to describe remote functions this server can handle.

```

fun main() {
    embeddedServer(Netty, port = 8080) {
        install(CallLogging)
        install(KRemote) {
            callableMap = CallableMap(..)
        }
        routing {
            remote("/call")
        }
    }.start(wait = true)
}

```

Figure 8: Kotlin Remote server

The problem is that server as opposed to client should not only serialize and deserialize function arguments and return value, but also invoke function itself. To do that RemoteCallable should be extended with field of type RemoteInvokator (figure 9). RemoteInvokator is basically a reference to the remote method. It is easier to understand by example on figure 10 with RemoteCallable for multiply function. Of course, this multiply function should have a normal body and no context parameters as opposed to the client version.

Now server can invoke RemoteInvokator when it receives RemoteCall. To do that server finds RemoteCallable in CallableMap by callableName from RemoteCall.

3.1.3 Single codebase

Described methods for creating client and server are not very good, when client and server share the same codebase. By that we mean that client and server code is in a sin-

```

data class RemoteCallable(
    val returnType: RemoteType,
    val invokator: RemoteInvokator,
    val parameters: Array<RemoteParameter>,
)

fun interface RemoteInvokator {
    suspend fun call(parameters: Array<Any?>): Any?
}

```

Figure 9: RemoteCallable with invokator

```

RemoteCallable(
    returnType = typeOf<Long>(),
    invokator = { multiply(it[0] as Long, it[1] as Long) },
    parameters = arrayOf(typeOf<Long>(), typeOf<Long>()),
)

```

Figure 10: RemoteCallable for multiply

gle Gradle module. Gradle [7] is the most popular build system used by Kotlin, Kotlin does not have its own concept of modules. The idea is that using the same function for client calls and server handling is more convenient than using two functions with the same signature. However, bodies of the remote functions are different for client point of view and server point of view. In order to fix that we can use more elaborate context parameters. Basically a "switch" between server and client body of the remote function is needed, and such switch can be encoded using the same context parameter that was used for remote call. Take a look at modified version of multiply function in figure 11. RemoteContext is basically a sum of types, it is either LocalContext or ConfiguredContext. Notice how covariant type parameter T and type Nothing are used here. Because Nothing is a bottom type in Kotlin type system (in other words it is subtype of every type) and T is covariant, LocalContext is a subtype RemoteContext for all T. It means that any remote function can be called in a LocalContext, and it will type check. When remote function is called in a LocalContext it will be executed in a current coroutine and will not result in a request to the server. Because of that the same function can be called from server in a local context and from client but in configured context.

Now RemoteInvokator should be changed a bit to reflect this duality of remote functions. Added context parameter hints that remote function can be only called locally on a server. Now server have to provide LocalContext for all the invoker calls.

3.1.4 Remote context relations

Contexts of remote functions can be used to express quite complex relations between capabilities of different servers using subtyping. For example let us imagine that there are two servers for arithmetic operations. One server only provides basic operations like multiplication etc., while other also supports trigonometric functions. That can be expressed on a type level in the way shown on figure 13. Clients here form a hierarchy. Notice how mul and divide can be called from sin function. More than that, calls to mul and divide from sin will not result in a remote call, because everything will be done in LocalContext provided by the invoker on the server. However, if user really wants to

```
sealed interface RemoteContext <out T: RemoteConfig>
object LocalContext: RemoteContext<Nothing>
class ConfiguredContext<T: RemoteConfig>(val config: T): RemoteContext<T>

context(ctx: RemoteContext<RemoteConfig>)
suspend fun multiply(lhs: Long, rhs: Long) =
    if (ctx is LocalContext) {
        lhs * rhs
    } else {
        (ctx as ConfiguredContext<RemoteConfig>).config.client.call<Long>(
            RemoteCall("multiply", arrayOf(lhs, rhs))
        )
    }
}
```

Figure 11: Updated multiply

```
fun interface RemoteInvokator {
    context(_: LocalContext)
    suspend fun call(parameters: Array<Any?>): Any?
}
```

Figure 12: Updated RemoteInvokator

call functions marked with ArithmeticConfig from inside of sin remotely, he can change context by calling `context(ArithmeticConfigImpl) { ... }`.

```
interface ArithmeticConfig: RemoteConfig
object ArithmeticConfigImpl: ArithmeticConfig { ... } // http://localhost:8001
object TrigonometricConfig: ArithmeticConfig { ... } // http://localhost:8002
context(_: RemoteContext<ArithmeticConfig>)
suspend infix fun Double.mul(rhs: Double) = this * rhs
context(_: RemoteContext<ArithmeticConfig>)
suspend infix fun Double.divide(rhs: Double) = this / rhs
context(_: RemoteContext<TrigonometricConfig>)
suspend fun sin(x: Double): Double {
    var (term, sum, n) = Triple(x, x, 1.0)
    while (abs(term) > 1e-5) {
        val divisor = (2.0 mul n) mul (2.0 mul n + 1.0)
        term = term mul -1.0 mul x mul x divide divisor
        sum = sum + term
        n++
    }
    return sum
}
```

Figure 13: Hierarchy of clients

3.2 Remote functions code generation

Some of the actions required from user in order to use remote functions can be delegated to the compiler using code generation. Namely, `CallableMap` and calls to contexts of

remote functions can be generated by the compiler. There are several standard ways to generate code in Kotlin. Kotlin Remote uses the most powerful one: compiler plugins.

Kotlin compiler architecture consists of two main parts: backend and frontend. Kotlin compiler plugins can also be separated into two categories: frontend plugins and backend plugins. Frontend plugins allow extending behavior of Kotlin compiler frontend. Backend plugins do the same thing but with compiler's backend. Both compiler's frontend and backend consists of the series of so-called lowerings, transformations of internal representation (IR) of the Kotlin source code in compiler. Plugins allow adding new lowerings, some of the lowerings do not change IR and only checks for some invariants.

The main difference between frontend and backend plugins is that frontend plugins only allow adding new definitions and change attributes of the old ones, but they do not allow changing bodies of definitions. However, results of plugins' work on frontend can be seen inside IDE, making frontend a great place to add verification, like checking that all the remote functions are suspend. Some code generation can also be done on frontend easier than on backend. Because of that Kotlin Remote generate code on both frontend and backend. It is also worth noticing that backend of the Kotlin compiler have less stable API. So it is generally a good idea to make changes on frontend whenever it is possible.

Nevertheless, Kotlin Remote plugin generates almost all the code in backend plugins, because they are much more powerful. This is why from now on we will focus mainly on backend part of the Kotlin Remote compiler plugin.

3.2.1 Remote function body generation

Firstly, there is a small problem with using compiler plugin. Now it is hard to distinguish remote functions from normal ones. Of course, context parameter can be used to do that, but it is a bad approach because not every function with context parameter of type `RemoteContext` should be remote. Sometimes we want to add context parameter to a function to mark that this function can call remote functions. But it shouldn't mean that the function itself is remote. Let us take a look at example on figure 14. Implementation of `multiply` is from figure 11. Here `power` function has a context parameter of type `RemoteContext` but is not itself remote, its body is executed locally, but context parameter shows that `power` function is capable of calling remote functions like `multiply`. Basically context parameter on `power` function signals that it has a *coeffect*, in other words it requires from environment a way to make remote calls.

To express such coeffects we need a separate way to mark remote functions. In Kotlin Remote annotation `Remote` is used for that. On figure 14 you can notice that `multiply` function has such annotation. Every function marked with `Remote` undergoes a body transformation that is exactly the same with transformation of `multiply` function that was done manually on figure 11. Name of remote call is a fully qualified name of the function. It is worth noticing that function marked with `@Remote` annotation has to have context parameter of type `RemoteContext`, otherwise error will be raised by Kotlin Remote frontend compiler plugin. This error will be visible in IntelliJ IDEA.

3.2.2 Callable map generation

In the end of section 3.1.1 we said that there is a problem with user having manually describe `CallableMap`. Kotlin Remote addresses this issue by automatically generat-

```

@Remote
context(_: RemoteContext<RemoteConfig>)
suspend fun multiply(lhs: Long, rhs: Long) = ...

context(_: RemoteContext<RemoteConfig>)
private suspend fun power(base: Long, power: Int): Long {
    return generateSequence { base }
        .take(power)
        .fold(1L) { acc, x -> multiply(acc, x) }
}

```

Figure 14: Local function with RemoteContext

ing callable map using function called `genCallableMap` from figure 15. As you can see this function basically has no body. This is because every call to this function is substituted with `CallableMap` instance generated by compiler plugin. One can say that `genCallableMap` is basically a macro (like in C). But a very smart macro. Kotlin Remote compiler plugin scan the source code during compilation, finds each function marked with `@Remote` and for each such function generates entry in `CallableMap` describing this function. All the information needed is accessible during compile time. This means that callable map generated `genCallableMap` can have lots and lots of entries, because functions from all over the source code are gathered.

It is worth noticing that `genCallableMap` adds to callable map even private, protected and internal functions. This means that by using `genCallableMap` it is possible to invoke such functions effectively breaking encapsulation. So it is recommended to keep all the remote functions public. Another quirk about `genCallableMap` is that it will not generate entries from remote functions from dependencies, because source code of dependencies is not accessible (it is already compiled). To alleviate that dependencies can save their callable maps generated by `genCallableMap`, for example, in a public static property. This way dependent code can use these saved callable maps by, for example, merging them with its own callable map.

```

fun genCallableMap(): CallableMap =
    throw IllegalStateException("Intrinsic function was called directly")

```

Figure 15: `genCallableMap` function

3.3 Exception handling

During executing of remote functions on remote machines exceptions can occur. Such erroneous situations should be handled by the client that made remote call. Here a tough decision should be made. We basically have two options: hide the fact that exception occurred on remote machine or make it obvious. Both options have its benefits and drawbacks, but it was decided to opt for the second approach, because we wanted to make calls to remote functions more seamless. So in the current implementation exception thrown on remote machine can be caught on the client as if function was called locally. Take a look at listing 16. This code will output exception from listing 1. You can see that there is now frames in exception related to remote call except synthetic stack frame `== Remote Call Boundary ==` that indicates a boundary between stack frames from remote machine and local frames.


```

@Remote
context(_: RemoteContext<RemoteConfig>)
suspend fun div(x: Int, y: Int) = x / y

fun main() {
    try {
        ServerConfig.runWith { div(10, 0) }
    } catch (_: ArithmeticException) {
        e.printStackTrace()
    }
}

```

Figure 16: Remote exception handling

```

java.lang.ArithmeticException: / by zero
    at proposalExamples.ExceptionKt.div(Exception.kt:11)
    at == Remote Call Boundary ==.(Unknown Source)
    at proposalExamples.ExceptionKt.main(Exception.kt:15)
    ...

```

Listing 1: Remote exception content

To implement this at first remote function should be able to return either a result or an error. In Kotlin this can be expressed like on listing 2.

```

sealed interface RemoteResponse<T> {
    data class Success<T>(val value: T): RemoteResponse<T>
    data class Failure(val error: Throwable): RemoteResponse<Nothing>
}

```

Listing 2: Return type of remote functions

However, two more things should be implemented: serialization of exceptions and altering stack traces of exceptions. Unfortunately, exceptions are not serialized by default using `kotlinx.serialization`, a library that Kotlin Remote uses for serialization. To make exceptions serializable they should be registered as polymorphic subclasses of `Throwable`. This is already done for all exceptions from a standard Kotlin Multiplatform library. When user wants to throw custom exception, he should register it in a `SerializersModule` using `throwableSerializer` function like at listing 3. When exception is not registered it will be serialized as `UnregisteredRemoteException`. To alter stack traces in KMP expect/actual functions are used. Unfortunately only JVM backend allows to change exceptions. For all other backends exception from remote machine is wrapped in a `RemoteException` as a cause.

```

SerializersModule {
    polymorphic(Throwable::class) {
        subclass(MyException::class, throwableSerializer(::MyException))
    }
}

```

Listing 3: Custom exception registration

3.4 Architecture

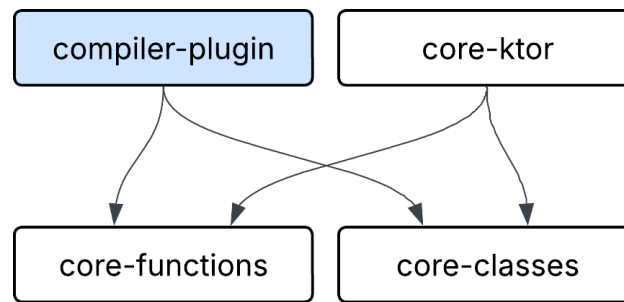


Figure 17: Module dependency graph

Now is a good time to take a look at architecture of kotlin-remote framework. At 17 is shown module dependency graph of the framework. Each rectangle is a Gradle module. Before we only described `compiler-plugin` and `core-functions` module. But there are two more modules that extend functionality of the mentioned ones. `core-classes` contain support for distributed objects and `core-ktor` provides an implementation of the transport protocol for Kotlin Remote using Ktor.

3.5 Ktor integration

Kotlin Remote is meant to be transport agnostic. This is done by splitting code into three modules: `core-functions`, `core-classes`, `core-ktor`. Until now, we almost exclusively have been discussing `core-functions` module that contains logic related to remote functions. Now let us briefly discuss `core-ktor` module which contains Kotlin Remote integration with Ktor. Ktor is a mature framework that offers lots of functions like auth support, logging etc. Kotlin Remote used together with Ktor supports the same functionality Ktor does. For example, authentication can be supported using code from listing 4. Notice call to `handleRemoteCall` function in `server`. This function passes control to `core-ktor` module that deserializes arguments of remote call, invokes remote function, serializes and sends result to the client. Inside `/call` route `handleRemoteCall` is called implicitly.

3.6 Remote classes

Because Kotlin is not only procedural and functional language but also relies heavily on OOP, it was decided to add support to call methods of the classes remotely, not only top-level functions. Basic support for that is achieved easily. The only important difference between top level functions and method function is that methods have implicit `this` parameter. In our case we can think about `this` parameter just like about any other regular parameter. So calling a remote method serializes `this` and other method parameters on client, sends call on sever, server deserializes `this` and other parameters and calls method on deserialized `this`. The good news are this is already supported without any additional effort because Kotlin compiler treats `this` as a regular parameter of the function. But there are two huge problems with this approach. Firstly, `this` should be serializable, and serialization logic should be provided by the user. Secondly, methods that mutate `this` will mutate deserialized copy of `this`, which basically means that they will have no effect on the original `this`.

All these problems are not new and were addressed in lots of existing RMI frameworks,

```

val server = embeddedServer(Netty, port = 8080) {
    install(Authentication) {
        basic("auth-basic") { validate { ... } }
    }
    install(KRemote) { callableMap = genCallableMap() }
    routing {
        authenticate("auth-basic") {
            post("/callAuth") {
                println("Welcome to the protected route!")
                handleRemoteCall()
            }
        }
        remote("/call")
    }
}

data object AuthServerConfig : RemoteConfig {
    override val client: RemoteClient = HttpClient {
        ...
        install(Auth) { ... }
    }.remoteClient(genCallableMap(), "/callAuth")
}

```

Listing 4: Authentication support

like Java RMI. In Kotlin Remote we decided to use well-known approach to solve these problems: stubs. Classes in Kotlin Remote can be marked with `@RemoteSerializable` annotation. For each remote serializable class custom `kotlinx.serialization` serializer is generated.

This custom serializer serializes remote class as stub. Stub is a subclass of remote class that is generated for every remote class. Stub only has two fields: ID and URL of the machine it was created on. During serialization of remote class it is added to the special map called `RemoteInstancesPool`. Keys in this pool are stub IDs and values are original classes.

Stub is deserialized either as original class in case it is deserialized on the machine it was serialized on and `RemoteInstancesPool` contains it, or as just stub. To understand whether stub was serialized on a certain machine, URL of this machine is compared with URL from the stub.

This URL is only used for this disambiguation of the nodes and garbage collection (more about it later in [3.7](#)). This URL is not used to make remote calls at all, address of the target machine is still received from context parameter of remote functions. URL is specified on the server when application is configured. It means that user should know address of the machine the server is running on.

Basically stubs are "remote references" that are supported by Kotlin Remote without altering logic related to remote function. Only serialization functionality is changed. Because of that logic related to remote classes lives in a separate Gradle module named `core-classes` which neither depends nor is dependent on `core-functions` module de-

scribed earlier. It is worth noticing that stubs are serialized as is, this allows to pass remote references from one machine to another even if reference is created on a third machine.

3.6.1 Remote classes code generation

It was said that for each `@RemoteSerializable` class a stub subclass is generated. This subclass is generated inside original class like on listing 5. `RemoteClassStub` extends `Calculator` and implements interface `Stub`. `Stub` interface can be used by user to check whether a certain instance of `Calculator` a stub or not. Notice how `RemoteClassStub` invokes `Calculator`'s default constructor even though `Calculator` doesn't have one. This is achieved by generating default constructor for `Calculator` in Kotlin Remote backend compiler plugin. `Calculator` is also made open implicitly by the same plugin on backend. Because these changes are made on backend, `Calculator` class cannot be extended, and its default constructor cannot be called from user defined Kotlin code. Only code generated by compiler plugin can do that.

```
@RemoteSerializable
class Calculator(private var init: Int) {
    @Remote
    context(_: RemoteContext<RemoteConfig>)
    suspend fun multiply(x: Int): Int {
        init *= x
        return init
    }
    class RemoteClassStub(
        override val id: Long,
        override val url: String
    ) : Calculator(), Stub
}
```

Listing 5: Remote class stub

Unfortunately, without reflection (and we cannot use it because of KMP) it is unclear how to deserialize `Calculator` as its `RemoteClassStub` because every remote class has its own `RemoteClassStub` type. To solve this issue another Kotlin Remote intrinsic is used, called `genRemoteClassList`. Take a look at listing 6. List of `RemoteClassDescriptor` is used to generate serializers and deserializers for remote classes using contextual serialization from `kotlinx.serialization`.

```
data class RemoteClassDescriptor<T : Any>(
    val clazz: KClass<T>,
    val serialName: String,
    val stubFabric: (Long, String) -> T,
)

fun genRemoteClassList(): List<RemoteClassDescriptor<Any>> = RemoteIntrinsic
```

Listing 6: `genRemoteClassList` intrinsic

3.7 Garbage collection

For now we described no way to remove remote classes from `RemoteInstancesPool`. This is a serious resource leak: every time a remote class is serialized on the server a new entry is added to the pool and it stays there forever. To resolve this issue several techniques can be used, including reference counting or full-blown distributed garbage collection. But we decided to opt for leasing approach, because it is easy, lightweight and durable. The general idea is that on the server every pooled instance has an associated *lease* with expiration time. Clients that hold a stub for the instance periodically renew this lease. When the lease expires the server removes the instance from the pool. The algorithm is similar to the one used in Java RMI distributed garbage collection [9].

On the server side `LeaseManager` keeps current expiration time per instance together with a set of client identifiers that currently hold leases on it. The latter is needed because the same instance can be referenced from several clients at once. Each renewal pushes the expiration time forward, and instance is removed only when none of its clients renews the lease for longer than the configured lease duration plus a small grace period for clock skew. A background coroutine periodically scans the leases and removes expired entries from `RemoteInstancesPool`. Default lease duration is thirty seconds, scan runs every ten seconds.

On the client side tracked stubs are grouped by server URL and renewal requests are batched per URL. Each stub is held through a weak reference, so client-side garbage collector is free to reclaim it as soon as user code drops it. On every renewal cycle (ten seconds by default, which is below the server lease duration so a single missed renewal does not reclaim the instance) tracked entries are split into still-reachable and reclaimed, with renewal request sent for the first group and release request for the second. Release request lets the server drop the instance immediately if no other client holds it.

This gives reasonable behaviour for the usual failure cases for free. When a client crashes its lease just expires and the instance is removed. When the server restarts the client receives failed identifiers in the renewal response and notifies user code through `onLeaseExpired` callback. Transient network errors are absorbed because a missed renewal is retried on the next cycle. Garbage collection is enabled by default but can be disabled or fine-tuned through `LeaseConfig` and `LeaseRenewalClientConfig`. The leasing layer is transport-agnostic: core defines abstract `LeaseClient` interface, Ktor integration provides the default implementation with `/lease/renew` and `/lease/release` HTTP routes.

4 Evaluation of the Kotlin Remote

To evaluate the framework four example applications were developed using Kotlin Remote and rewritten using Kotlin RPC. All four applications follow shared-codebase model: client and server parts live in the same Kotlin project and share data types. This is the only setting Kotlin Remote is designed for, so the evaluation does not attempt to cover scenarios outside of it (for example, separate codebases or cross-language communication). All implementations are functionally equivalent, the only difference is in the way network interactions are organized. As we are interested in network code, we will be focusing on it during comparison. Business logic of applications (database access, command line user interface, etc.) is roughly the same in all implementations. Source code

is available at `integration-tests` folder of Kotlin Remote repository [25].

We compare Kotlin Remote only with Kotlin RPC. The reason is that for shared-codebase Kotlin only setting, which is the setting Kotlin Remote targets, gRPC and Kotlin RPC are very close in features: both rely on service interface declaration, both generate stubs and server bases at compile time, both require to implement the service interface. But Kotlin RPC is more concise and more idiomatic for Kotlin. It uses Kotlin types directly with no separate IDL, serialization is provided by `kotlinx.serialization`, remote functions are suspend functions. gRPC in comparison has separate proto IDL, generated builder-based types that do not look like normal Kotlin code, and per remote function overhead that includes proto messages and proto-to-domain mapping. In shared-codebase setting this proto layer is essentially duplicating domain types that are already shared. We did prototype gRPC versions for the smaller applications and the per function overhead in gRPC was always the largest of the three frameworks. So we drop gRPC from the main comparison and focus on the closer alternative. This is not a statement about gRPC in general, it is fully expected that gRPC pays this overhead in shared-codebase setting because the setting is not what it is designed for.

Table 1 summarizes how the three frameworks compare on the features that matter for choosing between them. gRPC and Kotlin RPC look similar on the API style rows: both require a service interface declaration, neither makes a remote call visible at the call site, neither supports stateful remote objects with framework-managed lifecycle, neither expresses hierarchies between services in the type system. They differ on cross-language support and on whether the transport is fixed. Kotlin Remote differs from both alternatives on the API style rows and on the absence of streaming, which it currently does not support.

Feature	gRPC	Kotlin RPC	Kotlin Remote
Service interface required	Yes (IDL)	Yes (<code>@Rpc</code>)	No
Top level functions	No	No	Yes
Call site visibility	No	No	Yes
Hierarchical context tiers	No	No	Yes
Stateful remote objects	No	No	Yes
Streaming	Yes	Yes (<code>Flow</code>)	No
Kotlin Multiplatform support	Limited	Yes	Yes
Cross-language	Yes	No	No
Transport	HTTP/2 (fixed)	Pluggable (Ktor)	Pluggable (Ktor)

Table 1: Feature comparison of three RPC frameworks for Kotlin.

Four applications were chosen so that they cover different scenarios. *Todo* is a simple CRUD application with four operations and baselines per application setup cost. *Rooms chat* is a chat with rooms that uses remote classes and peer-to-peer architecture. *Social platform* is a backend split into ten microservices with thirty-six remote operations to stress per microservice overhead and orchestration. *CMS* is a small content management system with four hierarchical context tiers and Ktor authentication, designed to exercise the hierarchical contexts feature of Kotlin Remote.

4.1 Example applications

4.1.1 Todo application

This is the smallest of the three applications. It allows user to create, update, delete and list todo items. State is persisted in H2 database accessed via Exposed ORM. Server exposes only four remote functions and these functions are top level functions, they are not methods of any class. Listing 7 shows the entire remote API of the Todo application written using Kotlin Remote.

```
@Remote
context(_: RemoteContext<ServerConfig>)
suspend fun createTodo(request: CreateTodoRequest): Todo =
    Dependencies.repository.create(request)

@Remote
context(_: RemoteContext<ServerConfig>)
suspend fun updateTodo(id: Long, request: UpdateTodoRequest): Todo =
    Dependencies.repository.update(id, request)

@Remote
context(_: RemoteContext<ServerConfig>)
suspend fun deleteTodo(id: Long) = Dependencies.repository.delete(id)

@Remote
context(_: RemoteContext<ServerConfig>)
suspend fun todos(): List<Todo> = Dependencies.repository.readAll()
```

Listing 7: Todo application remote API in Kotlin Remote

The client invokes these functions inside `context(ServerConfig.asContext()) { ... }` block and the framework takes care of the network call. There is also a desktop client built with Compose Multiplatform on top of the same API. From the UI code point of view, remote calls are just suspend functions that have to be invoked from inside of context block.

4.1.2 Rooms chat

The second application is a chat in which users can create rooms, enter and exit them, send messages and list available rooms. The application is more interesting because it uses remote classes feature: a `Room` is a remote class which lives on the central server, while users are represented as remote configurations pointing to user machines. Listing 8 shows the most important parts of the chat code.

The interesting moment is the body of broadcast. Each user has its own `remoteContext` pointing to its machine. Inside `broadcast` the central server switches context to each user in turn and calls `send` function. This results in remote call from the central server to each user machine. So in the rooms chat application messages are delivered using genuine peer-to-peer remote calls instead of streaming. Each user runs an embedded remote server on its machine in addition to the central server. This kind of architecture is hard to express using more traditional RPC frameworks, because they assume strict

```

@RemoteSerializable
class Room private constructor() {
    private val clients = mutableListOf<User>()

    @Remote
    context(_: RemoteContext<RemoteConfig>)
    suspend fun enter(userName: String) { /* add user */ }

    @Remote
    context(_: RemoteContext<RemoteConfig>)
    suspend fun broadcast(fromUserName: String, message: String) {
        clients.filter { it.name != fromUserName }.forEach {
            context(it.remoteContext) { send("$fromUserName: $message") }
        }
    }

    companion object {
        @Remote
        context(_: RemoteContext<RemoteConfig>)
        suspend operator fun invoke(name: String): Room { ... }
    }
}

@Remote
context(_: RemoteContext<RemoteConfig>)
suspend fun send(message: String) { println(message) }

```

Listing 8: Rooms chat: Room class and send function

client-server relationship. In Kotlin Remote any participant can be both client and server because remote configuration is just a value that can be passed around.

4.1.3 Social platform

A small social network backend split into eight microservices: users, posts, comments, likes, follows, feed, notifications and search. On top of these two Backend-for-Frontend microservices are added: `webBff` returns rich aggregated objects for a desktop browser, `mobileBff` returns slim paginated objects for a mobile application. Total is ten microservices and thirty-six remote operations. The first six microservices keep their own state in memory, the other four are pure orchestrators that aggregate data from other microservices. The application was deliberately designed to stress per-microservice overhead and orchestration. Dependencies of every microservice are wired with Koin [12], one Koin module per microservice.

All ten services use no remote classes. Every remote operation is a top level function annotated with `@Remote`, and all thirty-six of them live in a single `API.kt` file. Listing 9 shows the `getFeed` function from the `feed` microservice, which uses five other microservices to assemble a feed.

To call a function from another microservice it is enough to wrap the call into context


```

@Remote context(_: RemoteContext<FeedServiceConfig>)
suspend fun getFeed(userId: Long): List<FeedItem> {
    val followees = context(FollowServiceConfig.asContext()) {
        getFollowing(userId)
    }
    val posts = followees.flatMap {
        context(PostServiceConfig.asContext()) { listUserPosts(it) }
    }
    return posts.map { p ->
        val author = context(UserServiceConfig.asContext()) {
            getUser(p.authorId)
        }
        val likes = context(LikeServiceConfig.asContext()) {
            getPostLikes(p.id)
        }.size
        val comments = context(CommentServiceConfig.asContext()) {
            getPostComments(p.id)
        }.size
        FeedItem(p, author, likes, comments)
    }
}

```

Listing 9: Service-to-service orchestration in social platform

block: context(XxxServiceConfig.asContext()) { ... }. There are no service stubs to inject, no constructor parameters to thread through and no startup ordering between microservices. BFF endpoints have the same shape as `getFeed`: switch context to each downstream microservice that is needed and assemble the response. In Kotlin RPC version of the application `getFeed` has to be a method of `FeedService` `@Rpc` interface, implemented by `FeedServiceImpl` class that takes five service stubs as constructor parameters (one per downstream microservice). The launcher of the `feed` microservice has to create those five stubs explicitly before starting the server. Each BFF microservice in Kotlin RPC pays the same cost: `@Rpc` interface, implementation class with four or five stub constructor parameters, launcher that builds the stubs in the right order.

Client side benefits from context parameters as well. Listing 10 shows the test function that exercises the platform. It declares context parameters for all ten services in a single `context(...)` block on its signature (we omitted some of the declarations). Inside the function every remote call is a plain unqualified function call, no per-call context block and no service stub. In Kotlin RPC version the same function takes ten service stubs as parameters and every call goes through the corresponding stub.

4.1.4 CMS

A small content management system designed specifically to exercise hierarchical contexts, the feature mentioned in section 3.1.1 but not otherwise tested in the previous applications. The CMS exposes fourteen remote operations split into four tiers of capabilities: Guest (anonymous read access), Author (create and edit own articles and comments), Moderator (delete any comment, pin article) and Admin (promote users, view statistics, purge user). In Kotlin Remote the four tiers are expressed as a chain of subtyped configs:


```

context(
    _: RemoteContext<UserServiceConfig>,
    _: <PostServiceConfig>,
    _: RemoteContext<FollowServiceConfig>,
    _: RemoteContext<LikeServiceConfig>,
    ...
)
suspend fun runTestClient() {
    val alice = register("alice", "pw", "Alice")
    val p1 = createPost(CreatePostDto(alice.id, "Hello #world"))
    likePost(p1.id, alice.id)
    val feed = getFeed(alice.id)
    val webHome = getWebHomePage(alice.id)
    val mobileHome = getMobileHomePage(alice.id, 0, 10)
}

```

Listing 10: Multi-context client function in social platform

```

interface GuestConfig : RemoteConfig
interface AuthorConfig : GuestConfig
interface ModeratorConfig : AuthorConfig
interface AdminConfig : ModeratorConfig

```

Listing 11: Four-tier context hierarchy in CMS

Each remote function declares the minimum tier it requires as its context parameter. Because of covariance and subtyping any function defined with a lower-tier requirement can be called from a higher-tier context. So a moderator session naturally calls guest-tier functions, and an admin session naturally calls functions of all three lower tiers. This propagation is in the type system, no extra wiring is needed at the call site.

Runtime protection of tiers is done with Ktor basic authentication. Server installs four basic auth providers, one per tier, each accepting credentials only if the authenticated user has at least the required role. Each tier is exposed under its own protected HTTP route (/call/guest, /call/author, /call/moderator, /call/admin). Each user session class points at the route matching its role and uses the user's credentials.

Hierarchical contexts show their value in cross-tier internal composition. Listing 12 shows the admin-tier `purgeUser` function which transitively calls moderator-tier and guest-tier functions to remove all articles and comments of a banned user.

All inner calls in `purgeUser` are remote functions of lower tiers, but on the server they execute locally with no network round trip. When `purgeUser` runs on the server the context provided by the framework is `LocalContext`, which is a subtype of `RemoteContext<X>` for any `X`, so the inner calls dispatch their local bodies. Kotlin RPC does not support hierarchy between `@Rpc` interfaces, so the equivalent project models the four tiers as four independent `@Rpc` interfaces with four implementation classes and four registered routes. Two consequences follow. First, the client has to obtain a separate stub for every tier it wants to call: a moderator client that needs to call a guest-tier function needs both a moderator stub and a guest stub. Second, cross-tier internal calls on the server cannot pass through the service abstraction without paying for a network round trip, so the Kotlin RPC version goes directly to the repositories from within the moderator and admin

```

@Remote context(_: RemoteContext<ModeratorConfig>)
suspend fun removeArticleAndComments(id: Long) {
    val comments = getComments(id) // requires GuestConfig
    comments.forEach {
        deleteAnyComment(it.id) // requires ModeratorConfig
    }
    dep<ArticleRepository>().deleteAny(id)
}

@Remote context(_: RemoteContext<AdminConfig>)
suspend fun purgeUser(login: String): Int {
    val articles = dep<ArticleRepository>().byAuthor(login)
    articles.forEach {
        removeArticleAndComments(it.id) // requires ModeratorConfig
    }
    val droppedComments = dep<CommentRepository>().deleteByAuthor(login)
    promoteUser(login, Role.GUEST) // requires AdminConfig
    return articles.size + droppedComments
}

```

Listing 12: Cross-tier internal composition in CMS

implementations, with an explanatory comment. This works but it bypasses the service layer.

4.2 Comparison to Kotlin RPC

Table 2 shows the size of framework-specific code for each application. By framework-specific code we mean code that exists only because the project uses this particular RPC framework's abstractions: remote function annotations and signatures (signatures count because they determine serialization), framework configuration classes and objects, `registerService` on the server and `withService` on the client (the analogs of "context object definition" and "context block opening" in Kotlin Remote), context blocks and remote calls at the call site, implementation classes that wrap `@Rpc` interfaces. The following code is excluded from our calculation:

1. Pure business logic (function bodies, repositories, data classes, CLI text)
2. Ktor transport setup: `HttpClient` and `embeddedServer`, installation of generic Ktor plugins, routing wrappers, JSON serializer configuration
3. Framework-specific transport-binding helpers: `install(KRemote)`, `install(KRpc)`, `rpc("/api")`, `remote("/call")`, `rpcConfig`, `installKRpc()`
4. Imports, package declarations and empty lines

Numbers are approximations because the split of some borderline lines is subjective.

Three patterns emerge from the table. For the Todo application both frameworks use the same amount of framework code (20 lines, 5 per operation). Kotlin Remote pays 3 lines per remote function (annotation with context parameter, then the suspend function signature on the next line, then the function body) while Kotlin RPC pays 2 lines per func-

Application	Kotlin Remote	Kotlin RPC	Per-op (KR / krpc)
Todo (4 ops)	20	20	5 / 5
Rooms chat [†]	~50	~35	—
Social platform (36 remote ops)	~155	~245	~4 / ~7
CMS (14 remote ops)	~75	~80	~5 / ~6

Table 2: Framework-specific code per application. [†]Rooms chat numbers are not directly comparable, see discussion below.

tion (interface signature plus impl override) plus around 5 lines per service for scaffolding (`@Rpc` interface declaration with braces, impl class declaration with `: ~ServiceName` and braces). For one service with four operations these cancel out exactly.

The CMS application has four tiers and fourteen operations and is essentially tied (around 75 versus 80). Kotlin Remote has four hierarchical `RemoteConfig` interfaces plus four session classes, which add up to roughly the same as the four `@Rpc` interfaces in Kotlin RPC. The hierarchical contexts feature does not by itself shrink the line count, the same information about tiers has to be expressed somewhere. What it changes is where the information lives: in Kotlin Remote it is in the type system and the compiler enforces it; in Kotlin RPC the programmer has to track at every call site which stub to use for which call. Cross-tier internal composition is local in Kotlin Remote (calls dispatch their local bodies because `LocalContext` is a subtype of `RemoteContext<X>` for any `X`), and either makes spurious network calls back to localhost or has to step around the service abstraction in Kotlin RPC.

The social platform shows the biggest gap (around 155 versus 245, Kotlin Remote is about 40 percent less). Per operation that is 4 lines for Kotlin Remote and 7 for Kotlin RPC. Two things contribute. First, ten microservices in Kotlin RPC mean ten implementation classes that mostly delegate to repositories (around 100 framework lines that do not exist in Kotlin Remote). Second, orchestrating microservices like `feed` and the BFFs require explicit stub creation (`withService` calls) for every dependent microservice, and matching `registerService` calls inside `AllLauncher`. The launchers in Kotlin RPC contribute around 43 framework lines compared to zero in Kotlin Remote. Kotlin Remote replaces both with one `API.kt` file of top level functions plus one `Configs.kt` with ten `RemoteConfig` objects.

Rooms chat numbers are not directly comparable because the two implementations do different things. In Kotlin Remote each user runs its own embedded server and message delivery is done as a remote call from the central server to the user machine using stored `RemoteContext` values. In Kotlin RPC the central server keeps a per user mailbox and pushes messages over a server side stream (`Flow<ChatMessage>`). Kotlin Remote version actually has more framework code than Kotlin RPC version (around 50 versus 35) because the peer-to-peer design needs two `RemoteConfig` classes (one per machine role) plus nine `@Remote` functions, against Kotlin RPC version's single `@Rpc` interface with eight methods.

4.2.1 Per-function overhead

Another way to compare frameworks is to count how much code is needed to add a new remote function to existing application. In Kotlin Remote it is just two extra lines: `@Remote`

annotation and `context(...)` declaration. Function body is the actual implementation. In Kotlin RPC each function has to be added in two places: once as a signature in the service interface and once as an override in the implementation class. Listing 13 shows the comparison.

```
// Kotlin Remote: 3 lines, no other file has to be modified
@Remote
context(_: RemoteContext<ServerConfig>)
suspend fun createTodo(request: CreateTodoRequest): Todo =
    repository.create(request)

// Kotlin RPC: signature in @Rpc interface + override in impl class
interface TodoService {
    suspend fun createTodo(request: CreateTodoRequest): Todo
}
class TodoServiceImpl(...) : TodoService {
    override suspend fun createTodo(request: CreateTodoRequest): Todo =
        repository.create(request)
}
```

Listing 13: Code required for one remote function

4.3 Limitations

4.3.1 Narrow scope of application

Kotlin Remote is optimized to work in a very specific environment: Kotlin only shared-codebase (monolithic) projects. This is the most important limitation of the framework and it is by design. Kotlin Remote does not try to be a general purpose RPC framework. It does not support cross-language communication, it is not meant for projects in which client and server are developed independently in separate codebases, and it is not meant for projects in which client and server have separate release cycles. In all of these scenarios general purpose frameworks like gRPC, Ktor or Kotlin RPC remain the correct choice and Kotlin Remote should not be considered as their replacement.

Several specific consequences follow from this scope decision. First, both client and server have to be written in Kotlin. Second, signatures of remote functions are shared between client and server through the common code of the project, so a change in remote function signature is a compile-time change for both sides at once. There is no notion of versioned API contract that clients can be incompatible with, this is left to the project's own release process. Third, the framework is not concerned with public-facing API ergonomics (stable URL design, custom HTTP headers, status codes), which are important for public REST APIs. Kotlin Remote endpoint is meant to be consumed by Kotlin clients from the same project, not by arbitrary third-party clients.

Within this scope Kotlin Remote gives substantial reduction of boilerplate, as shown in the previous sections. Outside this scope it is simply the wrong tool.

4.3.2 Dependency on experimental Kotlin features

Kotlin Remote uses context parameters which is experimental Kotlin feature at the moment of writing. User has to add `-Xcontext-parameters` compiler flag to enable it and the API of context parameters can change in future Kotlin releases. Kotlin Remote also has to be updated for every new Kotlin compiler version because the compiler plugin API is not stable.

4.4 Future work

Apart from addressing the limitations described in section 4.3, several directions for future work were identified during the development.

4.4.1 Code slicing

Currently every machine that runs Kotlin Remote application contains the full compiled artifact, including code that is meant to be executed on other machines. For the social platform this means that user microservice contains compiled code of all other microservices, even though it never executes them. Type information about target machine is already encoded in remote contexts (`UserServiceConfig`, `PostServiceConfig`, etc.). This information could be used by compiler plugin to slice the application into per node artifacts and include in each artifact only code reachable from that node. This is essentially what tierless languages discussed in section 2.7 do at the language level. Doing it as a compiler plugin pass over already typed Kotlin program is a feasible direction.

4.4.2 Remote lambdas

Kotlin Remote currently supports only named functions and methods of remote classes as remote callables. Lambdas (function values) are not supported. Adding remote lambdas would allow higher order remote functions like `remoteList.map { remoteFunction(it) }` where mapping is done on the remote machine. This would require to make function values serializable, which is a hard problem in general but seems tractable for closed-over values that are themselves serializable.

4.4.3 Bidirectional streaming

Adding streaming support, possibly via Kotlin `Flow` similarly to Kotlin RPC, would make Kotlin Remote suitable for applications like chats, push notifications and live updates without need to set up peer-to-peer architecture. Bidirectional streaming with backpressure is a natural fit for Kotlin coroutines.

5 Conclusions

In this thesis we developed Kotlin Remote, a lightweight RPC framework specifically for shared-codebase Kotlin Multiplatform projects. Kotlin Remote uses Kotlin's experimental context parameters feature to express remote calls. Context parameter of remote function carries information about target machine, which makes function callable on this machine, and at the same time appears in function signature, which makes remote calls clearly visible at the type level. This is a departure from the service-oriented RMI model adopted

by gRPC and Kotlin RPC, where remote calls are hidden behind service interfaces and at the call site look indistinguishable from local ones.

Kotlin Remote is designed for shared-codebase (monolithic) projects in which client and server live in a single Kotlin project, share data types and are released together. The framework is not meant to be a general purpose RPC solution and does not try to compete with gRPC, Kotlin RPC or Ktor outside of shared-codebase setting. Existing RPC frameworks are not boilerplate-heavy in general, their per function overhead is small. The service interface they require is a legitimate contract between client and server and is essential for cross-language projects, projects with independently developed client and server or projects with separately evolving public APIs. In all of these scenarios general purpose frameworks remain the correct choice. The contribution of this work is showing that in shared-codebase setting this service interface becomes redundant, because the contract is already implicit in the shared code, and removing this redundancy gives substantial reduction of code that the developer has to write.

The goal of the thesis stated in section 1.1 was to develop an RPC framework for shared-codebase Kotlin Multiplatform projects that reduces boilerplate code compared to existing alternatives in this setting. We believe this goal was reached.

Kotlin Remote has limitations that were discussed in section 4.3. The most important one is the narrow scope of application described in section 4.3.1. This narrow scope is intentional and follows from the goal of the thesis. Additionally, the framework at the moment of writing depends on experimental Kotlin features. Several directions for future work were identified during development, including per node code slicing, remote lambdas and bidirectional streaming.

References

- [1] Andrew D. Birrell and Bruce Jay Nelson. “Implementing remote procedure calls”. In: *ACM Trans. Comput. Syst.* 2.1 (Feb. 1984), pp. 39–59. ISSN: 0734-2071. DOI: [10.1145/2080.357392](https://doi.org/10.1145/2080.357392). URL: <https://dl.acm.org/doi/10.1145/2080.357392> (visited on 02/07/2026).
- [2] Adam Chlipala. “Ur/Web: a simple model for programming the web”. In: *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. POPL ’15. New York, NY, USA: Association for Computing Machinery, Jan. 2015, pp. 153–165. ISBN: 978-1-4503-3300-9. DOI: [10.1145/2676726.2677004](https://doi.org/10.1145/2676726.2677004). URL: <https://doi.org/10.1145/2676726.2677004>.
- [3] *Coeffects: Context-aware programming languages*. URL: <https://tomasz.net/coeffects/> (visited on 10/15/2025).
- [4] Ezra Cooper et al. “Links: web programming without tiers”. In: *Proceedings of the 5th international conference on Formal methods for components and objects*. FMCO’06. Berlin, Heidelberg: Springer-Verlag, 2007, pp. 266–296. ISBN: 978-3-540-74791-8. DOI: [10.1007/978-3-540-74792-5_12](https://doi.org/10.1007/978-3-540-74792-5_12). URL: https://doi.org/10.1007/978-3-540-74792-5_12.
- [5] *Coroutines | Kotlin*. en. URL: <https://kotlinlang.org/docs/coroutines-overview.html> (visited on 04/15/2026).
- [6] Roy Thomas Fielding. “Architectural styles and the design of network-based software architectures”. AAI9980887. PhD thesis. University of California, Irvine, 2000.
- [7] *Gradle Build Tool*. en-US. Feb. 2026. URL: <https://gradle.org/> (visited on 02/13/2026).
- [8] *gRPC*. en. URL: <https://grpc.io/> (visited on 02/07/2026).
- [9] *Java Development Kit Version 17 API Specification*. en. URL: <https://docs.oracle.com/en/java/javase/17/docs/specs/rmi/index.html> (visited on 02/10/2026).
- [10] *KEEP/notes/code-coloring.md at master · Kotlin/KEEP*. URL: <https://github.com/Kotlin/KEEP/blob/master/notes/code-coloring.md> (visited on 10/15/2025).
- [11] *KEEP/proposals/KEEP-0367-context-parameters.md at main · Kotlin/KEEP*. URL: <https://github.com/Kotlin/KEEP/blob/main/proposals/KEEP-0367-context-parameters.md> (visited on 10/15/2025).
- [12] *Koin: a pragmatic lightweight dependency injection framework for Kotlin*. en. URL: <https://insert-koin.io/> (visited on 04/15/2026).
- [13] *Kotlin Multiplatform*. en. URL: <https://kotlinlang.org/docs/multiplatform.html> (visited on 04/15/2026).
- [14] *Kotlin/kotlinx.rpc*. URL: <https://github.com/Kotlin/kotlinx.rpc> (visited on 04/15/2026).
- [15] *Kotlin/kotlinx.serialization*. original-date: 2017-07-20T11:25:23Z. Feb. 2026. URL: <https://github.com/Kotlin/kotlinx.serialization> (visited on 02/11/2026).
- [16] *Ktor: Build Asynchronous Servers and Clients in Kotlin*. en. URL: <https://ktor.io/> (visited on 02/11/2026).
- [17] Daan Leijen. *Koka: a Functional Language with Effects*. URL: <https://koka-lang.github.io/> (visited on 04/15/2026).

- [18] Jeffrey R. Lewis et al. “Implicit parameters: dynamic scoping with static types”. In: *Proceedings of the 27th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. POPL '00. New York, NY, USA: Association for Computing Machinery, Jan. 2000, pp. 108–118. ISBN: 978-1-58113-125-3. DOI: [10.1145/325694.325708](https://doi.org/10.1145/325694.325708). URL: <https://doi.org/10.1145/325694.325708> (visited on 04/15/2026).
- [19] Tomas Petricek, Dominic Orchard, and Alan Mycroft. “Coeffects: unified static analysis of context-dependence”. In: *Proceedings of the 40th international conference on Automata, Languages, and Programming - Volume Part II*. ICALP'13. Berlin, Heidelberg: Springer-Verlag, July 2013, pp. 385–397. ISBN: 978-3-642-39211-5. DOI: [10.1007/978-3-642-39212-2_35](https://doi.org/10.1007/978-3-642-39212-2_35). URL: https://doi.org/10.1007/978-3-642-39212-2_35 (visited on 02/09/2026).
- [20] František Plášil and Michael Stal. “An architectural view of distributed objects and components in CORBA, Java RMI and COM/DCOM”. en. In: *Software - Concepts & Tools* 19.1 (Mar. 1998), pp. 14–28. ISSN: 1432-2188. DOI: [10.1007/s003780050004](https://doi.org/10.1007/s003780050004). URL: <https://doi.org/10.1007/s003780050004> (visited on 02/07/2026).
- [21] Gordon Plotkin and Matija Pretnar. “Handlers of algebraic effects”. In: *Proceedings of the 18th European Symposium on Programming Languages and Systems: Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2009*. ESOP '09. Berlin, Heidelberg: Springer-Verlag, Mar. 2009, pp. 80–94. ISBN: 978-3-642-00589-3. DOI: [10.1007/978-3-642-00590-9_7](https://doi.org/10.1007/978-3-642-00590-9_7). URL: https://doi.org/10.1007/978-3-642-00590-9_7.
- [22] *Protocol Buffers*. en. URL: <https://protobuf.dev/> (visited on 04/15/2026).
- [23] Louis Ryan|. *gRPC Motivation and Design Principles*. en. Section: blog. Sept. 2015. URL: <https://grpc.io/blog/principles/> (visited on 02/10/2026).
- [24] *Servant - haskell-servant*. URL: <https://www.servant.dev/> (visited on 02/08/2026).
- [25] Aleksandr Stupnikov. *enotvtapke/kotlin-remote*. original-date: 2025-11-30T22:59:26Z. May 2026. URL: <https://github.com/enotvtapke/kotlin-remote> (visited on 05/18/2026).
- [26] Andrew S Tanenbaum and Robbert van Renesse. “A Critique of the Remote Procedure Call Paradigm”. en. In: ().
- [27] Andrew S. Tanenbaum and Maarten van Steen. *Distributed Systems: Principles and Paradigms (2nd Edition)*. USA: Prentice-Hall, Inc., Sept. 2006. ISBN: 978-0-13-239227-3.
- [28] *The Opa Language*. URL: <http://opalang.org/> (visited on 10/15/2025).
- [29] *What Color is Your Function? – journal.stuffwithstuff.com*. URL: <https://journal.stuffwithstuff.com/2015/02/01/what-color-is-your-function/> (visited on 10/15/2025).
- [30] James E. White. “A high-level framework for network-based resource sharing”. In: *Proceedings of the June 7-10, 1976, national computer conference and exposition*. AFIPS '76. New York, NY, USA: Association for Computing Machinery, June 1976, pp. 561–570. ISBN: 978-1-4503-7917-5. DOI: [10.1145/1499799.1499878](https://doi.org/10.1145/1499799.1499878). URL: <https://dl.acm.org/doi/10.1145/1499799.1499878> (visited on 02/10/2026).

- [31] Ann Wollrath, Roger Riggs, and Jim Waldo. "A distributed object model for the java™ system". In: *Proceedings of the 2nd conference on USENIX Conference on Object-Oriented Technologies (COOTS) - Volume 2*. COOTS'96. USA: USENIX Association, June 1996, p. 17. (Visited on 02/07/2026).

A Note on Code coloring

The idea to use context of the function to determine where it should be called comes from so called "code coloring" described in Kotlin KEEP [10]. The term code coloring in the KEEP is not introduced formally, instead only intuition, of what code coloring is, is provided using examples. Such an intuition cannot be used to reason about "code coloring" from theoretical point of view.

During work on this thesis we developed better understanding of "code coloring" and in this appendix we want to try to give formal definition of it.

Code coloring consists of two aspects: type system and compiler support. The most clean example of code coloring are suspend functions in Kotlin. Functions with `suspend` keyword are basically an extension of Kotlin type system. Suspend function can be called only from other suspend functions, that gives a shape to the color. But the gist of suspend functions is how they are treated by compiler. Basically suspend function is a mark for the compiler to compile code of the function in a special way, in a CPS style as an automaton. For CPS suspend functions have to receive additional argument, continuation, thus constraints on the form of suspend functions I mentioned previously. We can say that bodies of suspend functions are treated differently from the rest of the code. Basically, we can think about suspend functions as a separate programming language embedded in Kotlin, at least compiler treats bodies of suspend functions this way.

Suspend keyword mechanism that dictates the shape of suspend functions can be also expressed by requiring suspend functions to have specific signatures, like having specific return type or additional argument. And this is exactly what suspend functions do under the hood: accepting additional continuation argument. We could return something like `Deferred` from suspend functions, but it would require working with this type in a very explicit fashion, which basically eliminates the whole point of suspend functions in Kotlin. Good option for return type in languages with do-notations (like Haskell) is monad, because with do-notation we can work within monad without thinking of monad's specific type. In case of coroutines reader monad is enough because we already express callCC semantics using automata and CPS, thus we don't need monad's feature for explicit control flow (like in `Cont` monad). Reader monad can be expressed using context parameters we used for Kotlin remote.

So basically the problem of code coloring is the problem of embedded languages, because we want to embed language with its own compilation logic in host language. We are concentrating on embedding functions because it's a good separate part of code. With coroutines and Kotlin Remote language semantics inside suspend or remote function can be fully expressed by rewriting code by the compiler (from Kotlin to Kotlin). Thus we can apply one more restriction on code coloring: languages are embedded on the level of functions, which bodies should be compiled to host language, in our case Kotlin. To make it all easier we used Kotlin syntax even for embedded language. However, in common case it is not required. With Kotlin it is required because there is no way to change parsing pass in Kotlin compiler, we can only translate one syntactically correct code to another.

To give shape to colors using type we basically use coeffects as our theoretical basis. In Kotlin common functions are not pure and coeffect free at all, they require a lot from their environment, like a way to run threads, for example. So new "colors" can be introduced to be more restrictive than existing functions. In this case we cannot use types for colors,

because type can only extend, not restrict capabilities of basic functions. For such restrictive colors we can use special compilation of colored function to basically check that nothing illegal was called from inside of it. In Kotlin it can be done FIR compiler plugins for the most part.

Where color both extends capabilities of standard functions and restricts them, we can use compiler stage checks together with type system.

So colors are basically function level embedded languages, that use the same syntax as a host language and compiled to the host language. Compilers of such languages are basically translators.

Colors can be more restricted or more permissive than host language. To express more permissive colors that extend basic language we can use types (for example context parameters or monads). This is because to enter more permissive area of the language we need something from context, coeffect. To express restrictive colors we need to drop some capabilities from context, that can be done by compiler checks. For example, we can forbid loops in a certain functions to make this function straight line programs. To do that we need to at least forbid calling all the standard functions and allow calling only our marked, restricted functions. Marking can be done using annotations.

In Kotlin we have very powerful instruments to change compilation of certain functions (compiler plugins), but at the same time it very inconvenient. A good example where this process is powerful enough, but a lot more convenient is Rust with its procedural macros metaprogramming.

One more thing to notice is that in Kotlin Remote we basically use "parametrized colors" or "parametrized coeffects" (It is not official terms). While suspend function add single color, kotlin remote adds a whole bunch, but code translation for everyone of them is the same. The difference is only in what they require from environment, what specific coeffect. In our case, remote node url and so on. All the coeffects that are required from remote function are in the same group, thus the name "parametrized coeffects". So Kotlin Remote adds strictly speaking a bunch of similar colors, but this also can be expressed using type.

For restrictive colors such "parametrized colors" can also be expressed, using, for example, annotations with parameters.

At last definition for code color can be given. *Code color* — is a set of functions which bodies are translated (from host language to host language) during compilation in a special way, described by the color. Color consists of translator and a rules to describe colored set of functions. Rules consist of functions that entry a set. Colors applies restriction either on from what functions colored function can be called, what function can be called from colored ones or both. That depends on the color type.

Colors without translation are coeffect system. Colors without restrictions for colored functions are metaprogramming on function level (like Rust procedural macros). So alternative definition for *code coloring* — coeffect system with metaprogramming on function level.